

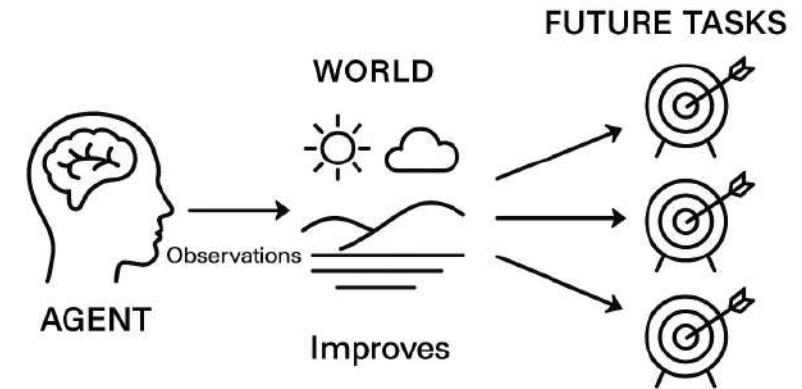


Learning From Examples

Md. Nurul Muttakin
Assistant Professor, CSE, BUET

Learning Agent

- An agent is **learning** if it improves its performance on future tasks after making observations about the world.



Learning Agent

- **Inductive Learning** → Specific data to Generic rule
- **Deductive Learning** → Less efficient rule to more efficient rule
- **Online Learning** → Continuously learning over-time
- **Offline Learning** → Pre-learned

Learning Agent

- **Components to be learned → ?**
 - Utility functions
 - Actions
 - Environments etc.
- **Representation and prior knowledge → ?**
 - Efficient prior knowledge representation
 - Efficient inference from prior knowledge
 - Reasoning based on a knowledge base

Learning Agent

Feedback to learn from

- No explicit feedback → ?
 - Unsupervised
- Explicit feedback → ?
 - Supervised
- Get rewarded or punished → ?
 - Reinforcement
- Internal feedback → ?
 - Self-supervised

Feedback to learn from



No explicit feedback → **Unsupervised**



Explicit feedback → **Supervised**



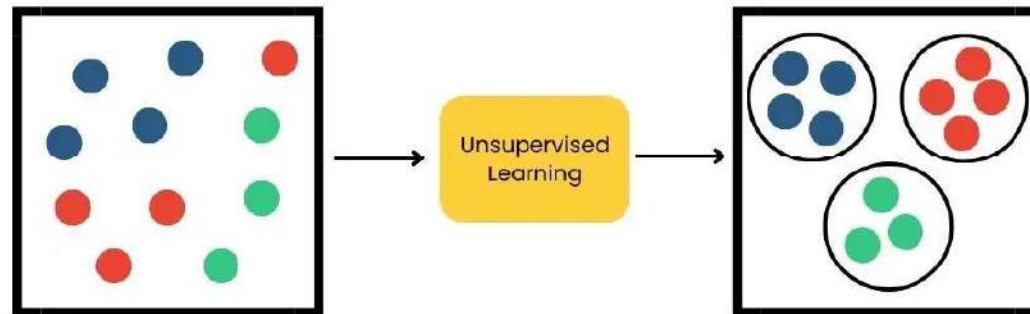
Get rewarded or punished → **Reinforcement**



Internal feedback → **Self-supervised**

Learning types

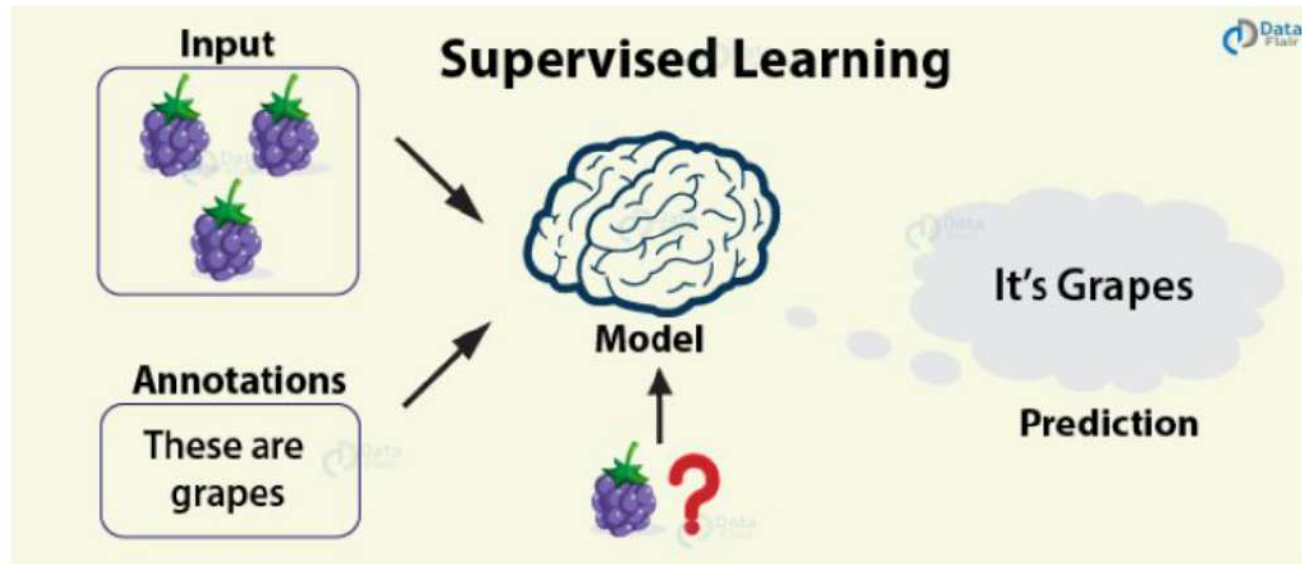
- **Unsupervised:** Learn patterns in the input
 - Example: “Good traffic days” and “bad traffic days” learned by a taxi driver without any label to a day
 - One common example, clustering
 - Data do not have labels



Learning types

■ Supervised:

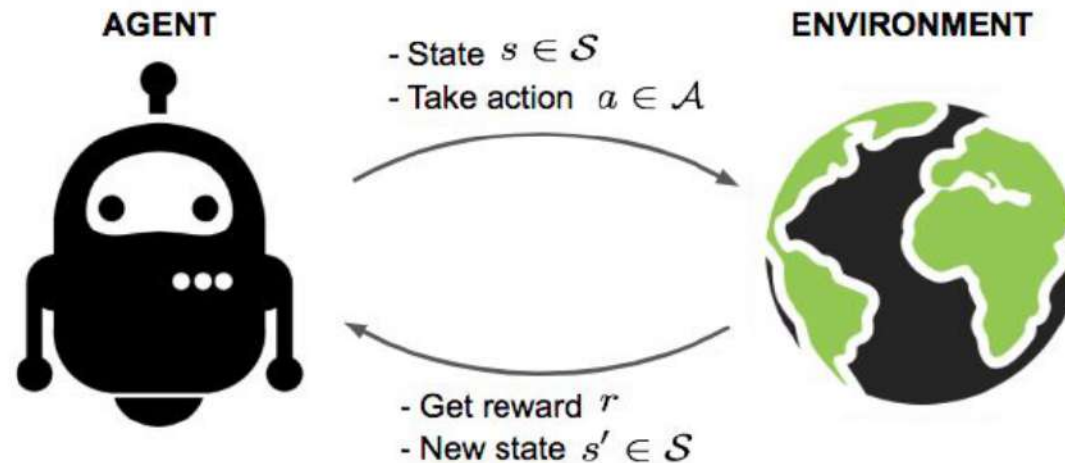
- Learn from input-output pairs.
- Data have labels (i.e., class)
- Can predict “Good” or “Bad” from traffic info of a day



Learning types

■ Reinforcement:

- Learn from reinforcements, rewards and punishments
- Example: lack of a tip at the end of a journey an indication that taxi driver did something wrong



Supervised Learning

- **Task:**

- Given a training set of N example input–output pairs $(x_1, y_1), (x_2, y_2), \dots (x_N, y_N)$,
- where each y_j (**ground truth / label**) was generated by an unknown function $y = f(x)$,
- discover a function h that approximates the true function f .

Supervised Learning

- **Task:**

- Discover a function h that approximates the true function $f(x)$.
- $x=[a, b, \dots]$ could be a vector and a, b , etc., are real numbers $\rightarrow$
Factored Representation of input
- If the range of $f(x)$ is a finite set $\rightarrow$ **classification problem** $\rightarrow$ Eg.,
Day type {rainy, cloudy, sunny}
- If the range of $f(x)$ is a number $\rightarrow$ **regression problem** $\rightarrow$ Eg.,
Temperature

Supervised Learning

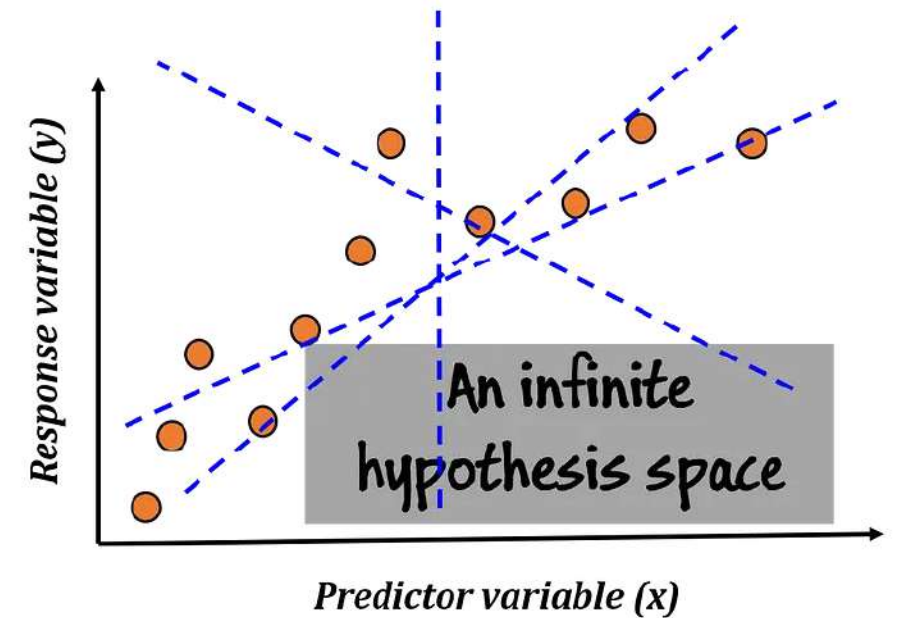
- **Task:**
 - Discover a function h that approximates the true function $f(x)$.
 - h is the approximator function
 - Approximator $\rightarrow$ Approximates the true function
 - Hypothesis $\rightarrow$ Hypothesizes on the true function
 - Predictor $\rightarrow$ Predicts the true function
 - Model $\rightarrow$ Models the true function

Supervised Learning

■ Task:

■ Searching for h in a

- Function class $H \rightarrow$ Approximator function h
- Hypothesis space $H \rightarrow$ Hypothesis h
- Predictor class $H \rightarrow$ Predictor h
- Model class $H \rightarrow$ Model h



Supervised Learning

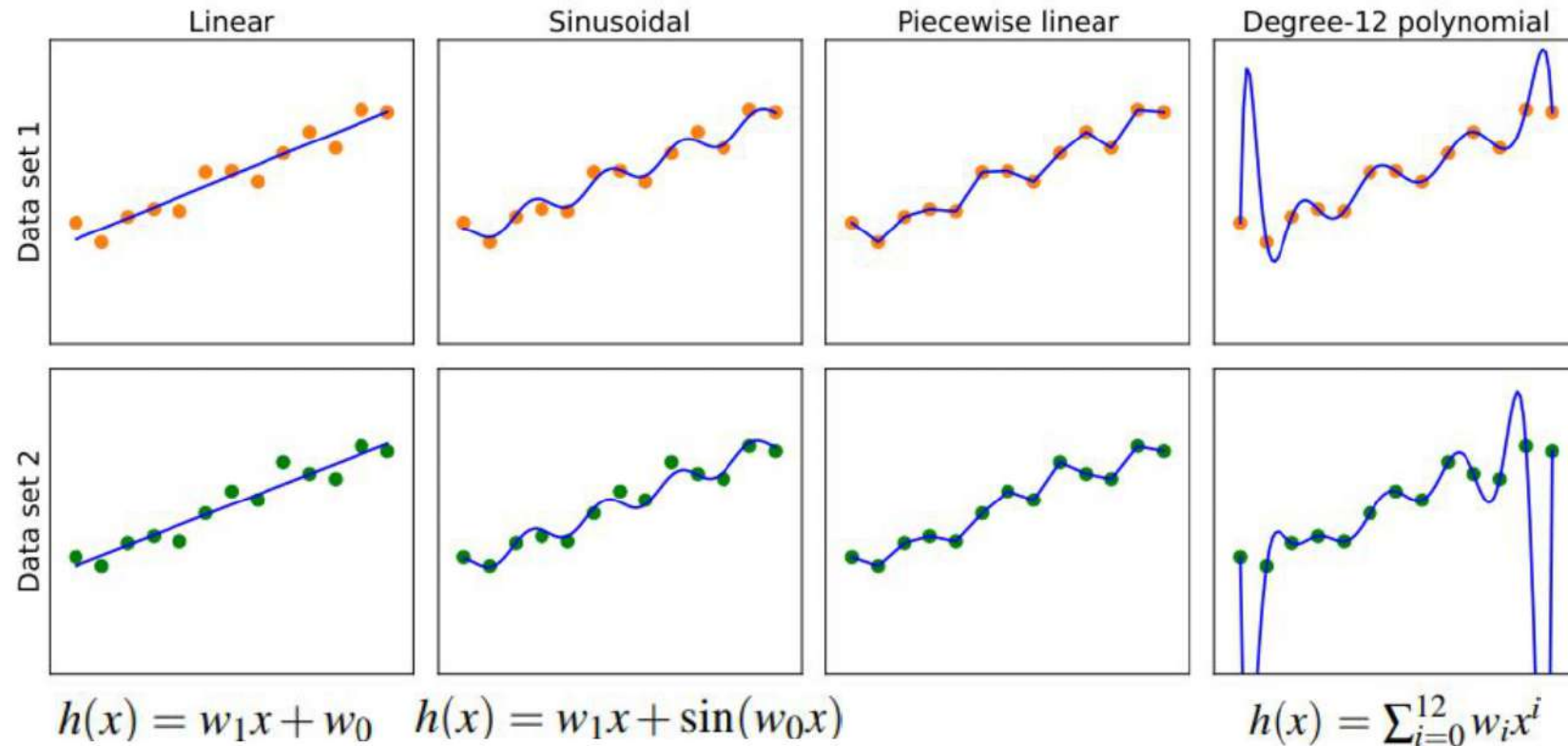
- **Hypothesis space:**
 - How do we choose a hypothesis space?
 - Prior knowledge about the problem to be learned.
 - If no idea, exploratory data analysis (EDA):
 - Examining the data with statistical tests
 - Visualizations—histograms, scatter plots, box plot, etc.

Supervised Learning

- **Hypothesis space:**
 - **Consistent hypothesis, h :** For each x_i in the training set has $h(x_i) = y_i$ (categorical value)
 - **Best-fit function, h :** For which each $h(x_i)$ is close to y_i (numerical value)

Supervised Learning

■ Hypothesis space:

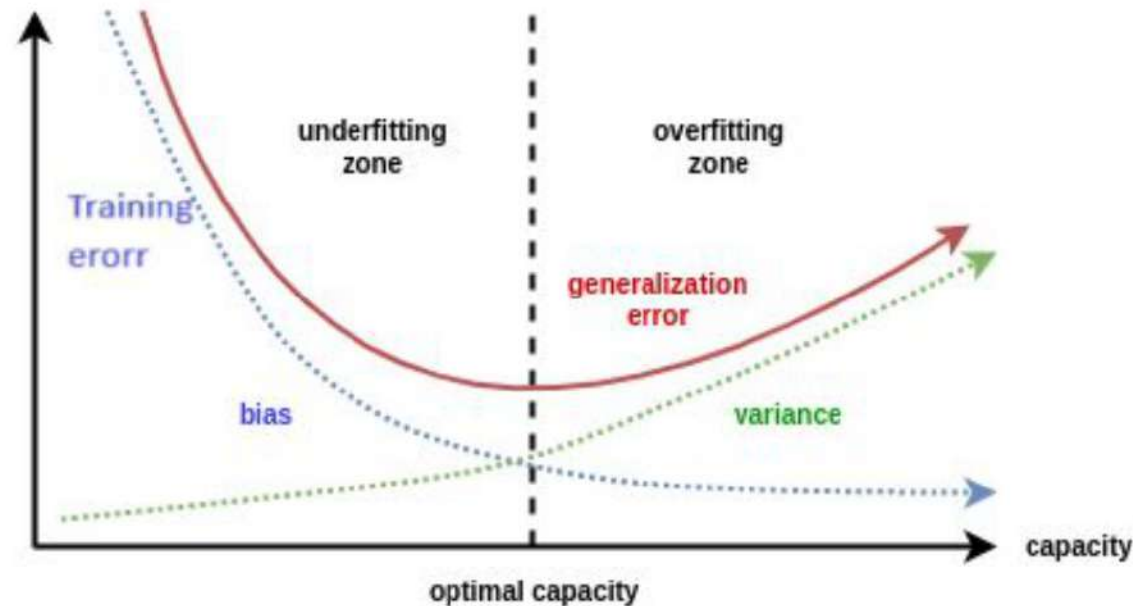


Supervised Learning

- **Measure the hypothesis, h :**
 - **Test set:** Novel samples not available during training
 - **Generalization:** Accurate prediction on test set
 - **Training Error/Loss:** Error in prediction on **training set**
 - **Generalization Error/Loss:** Error in prediction on **test set**

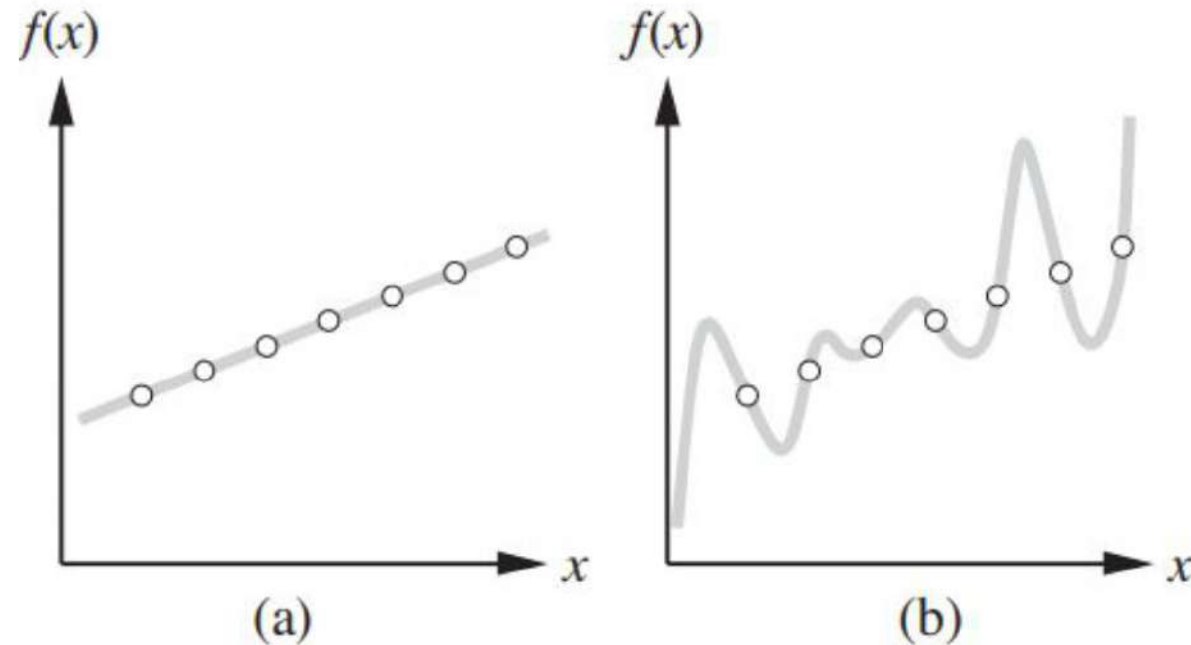
Supervised Learning

- Measure the hypothesis, h :
 - Less Training Error → Less bias → High consistency → Not underfitting
 - Less Test Error: → Less variance → High generalization → Not overfitting



Supervised Learning

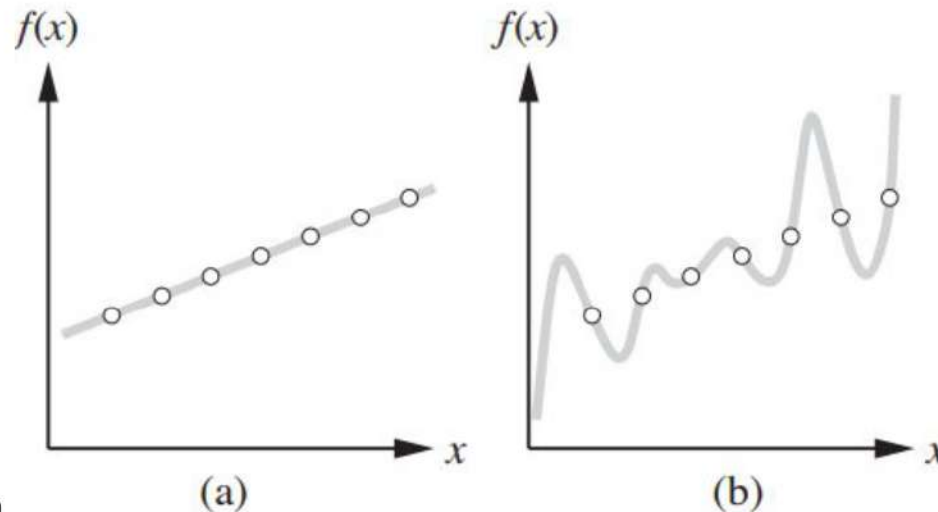
- Which hypothesis, h , to choose?:
 - Many consistent hypothesis



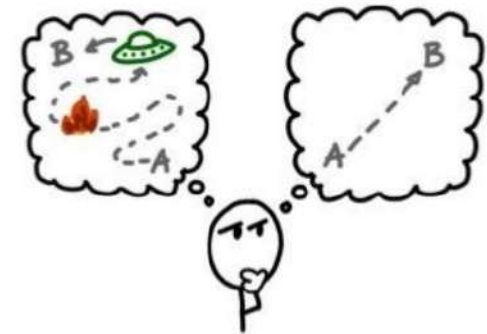
Supervised Learning

■ Which hypothesis, h , to choose?:

- **Ockham's razor:** Select the **simplest** hypothesis **consistent** with the data
- **Simplicity:** How to define? How to measure?
- Simplicity or Appropriateness?



Occam's Razor



"When faced with two equally good hypotheses, always choose the simpler."

Supervised Learning

- **Most probable hypothesis, h^* :**

- Choosing the hypothesis h^* that is most probable given the data

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(h|data) .$$

By Bayes' rule this is equivalent to

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(data|h) P(h) .$$

Supervised Learning

■ Most probable hypothesis, h^* :

- $P(h)$ = **Prior** probability that h holds before seeing the training data
- $P(data)$ = **Prior** probability of observing the training data
- $P(data|h)$ = (**Likelihood**) Probability of observing $data$ in a world where h holds
- $P(h|data)$ = (**Posterior**) Probability of h holding given observed $data$

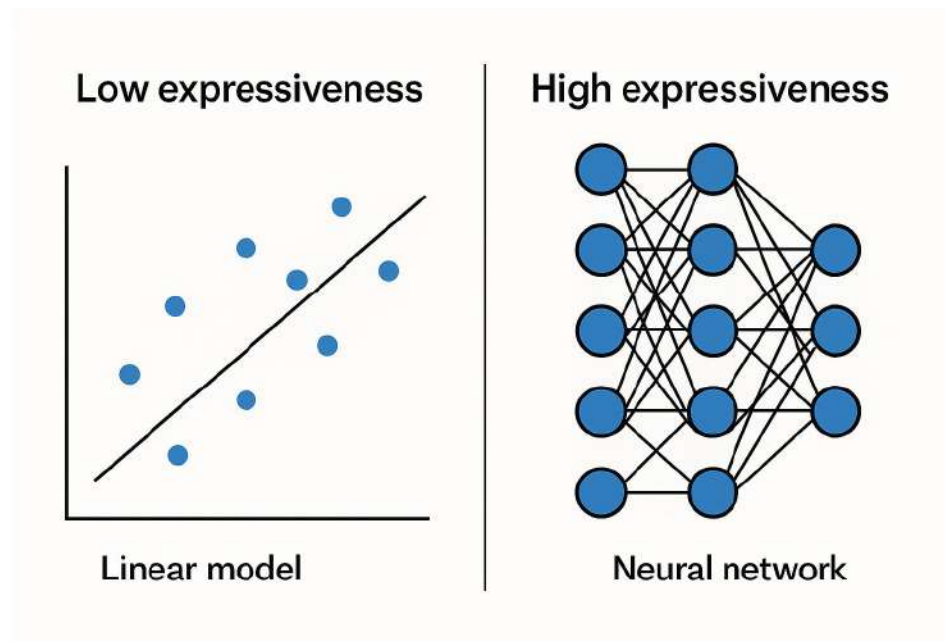
$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(h|data).$$

Prove that By Bayes' rule this is equivalent to

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(data|h) P(h).$$

Supervised Learning

- **Hypothesis space:**
 - **Expressiveness:** How rich or flexible the hypothesis space is
 - How many different functions or patterns it can represent
 - How **complex** functions it can represent



Supervised Learning

- **Model Class / Hypothesis Space:**
 - Decision Trees
 - Linear Models
 - Non-parametric Models: Nearest Neighbors
 - Ensemble Models: Random Forest etc.
 - Neural Networks

Formal Statistical Learning Model

- **The learner's input:** In the basic statistical learning setting, the learner has access to the following:
 - **Domain set:** An arbitrary set, $\mathcal{X}$. This is the set of objects that we may wish to label. For example, in the papaya learning problem mentioned before, the domain set will be the set of all papayas. Usually, these domain points will be represented by a vector of *features* (like the papaya's color and softness). We also refer to domain points as *instances* and to $\mathcal{X}$ as instance space.
 - **Label set:** For our current discussion, we will restrict the label set to be a two-element set, usually $\{0, 1\}$ or $\{-1, +1\}$. Let $\mathcal{Y}$ denote our set of possible labels. For our papayas example, let $\mathcal{Y}$ be $\{0, 1\}$, where 1 represents being tasty and 0 stands for being not-tasty.

Formal Statistical Learning Model

- **Training data:** $S = ((x_1, y_1) \dots (x_m, y_m))$ is a finite sequence of pairs in $\mathcal{X} \times \mathcal{Y}$: that is, a sequence of labeled domain points. This is the input that the learner has access to (like a set of papayas that have been tasted and their color, softness, and tastiness). Such labeled examples are often called *training examples*. We sometimes also refer to S as a *training set*.¹
- **The learner's output:** The learner is requested to output a *prediction rule*, $h : \mathcal{X} \rightarrow \mathcal{Y}$. This function is also called a *predictor*, a *hypothesis*, or a *classifier*. The predictor can be used to predict the label of new domain points. In our papayas example, it is a rule that our learner will employ to predict whether future papayas he examines in the farmers' market are going to be tasty or not. We use the notation $A(S)$ to denote the hypothesis that a learning algorithm, A , returns upon receiving the training sequence S .

Formal Statistical Learning Model

- **A simple data-generation model** We now explain how the training data is generated. First, we assume that the instances (the papayas we encounter) are generated by some probability distribution (in this case, representing the environment). Let us denote that probability distribution over $\mathcal{X}$ by $\mathcal{D}$. It is important to note that we do not assume that the learner knows anything about this distribution. For the type of learning tasks we discuss, this could be any arbitrary probability distribution. As to the labels, in the current discussion we assume that there is some “correct” labeling function, $f : \mathcal{X} \rightarrow \mathcal{Y}$, and that $y_i = f(x_i)$ for all i . This assumption will be relaxed in the next chapter. The labeling function is unknown to the learner. In fact, this is just what the learner is trying to figure out. In summary, each pair in the training data S is generated by first sampling a point x_i according to $\mathcal{D}$ and then labeling it by f .

Formal Statistical Learning Model

- **Measures of success:** We define the *error of a classifier* to be the probability that it does not predict the correct label on a random data point generated by the aforementioned underlying distribution. That is, the error of h is the probability to draw a random instance x , according to the distribution $\mathcal{D}$, such that $h(x)$ does not equal $f(x)$.

Formally, given a domain subset,² $A \subset \mathcal{X}$, the probability distribution, $\mathcal{D}$, assigns a number, $\mathcal{D}(A)$, which determines how likely it is to observe a point $x \in A$. In many cases, we refer to A as an event and express it using a function $\pi : \mathcal{X} \rightarrow \{0, 1\}$, namely, $A = \{x \in \mathcal{X} : \pi(x) = 1\}$. In that case, we also use the notation $\mathbb{P}_{x \sim \mathcal{D}}[\pi(x)]$ to express $\mathcal{D}(A)$.

We define the error of a prediction rule, $h : \mathcal{X} \rightarrow \mathcal{Y}$, to be

$$L_{\mathcal{D},f}(h) \stackrel{\text{def}}{=} \mathbb{P}_{x \sim \mathcal{D}}[h(x) \neq f(x)] \stackrel{\text{def}}{=} \mathcal{D}(\{x : h(x) \neq f(x)\}). \quad (2.1)$$

Formal Statistical Learning Model

- **A note about the information available to the learner** The learner is blind to the underlying distribution $\mathcal{D}$ over the world and to the labeling function f . In our papayas example, we have just arrived in a new island and we have no clue as to how papayas are distributed and how to predict their tastiness. The only way the learner can interact with the environment is through observing the training set.

Formal Statistical Learning Model

Empirical Risk Minimization

As mentioned earlier, a learning algorithm receives as input a training set S , sampled from an unknown distribution $\mathcal{D}$ and labeled by some target function f , and should output a predictor $h_S : \mathcal{X} \rightarrow \mathcal{Y}$ (the subscript S emphasizes the fact that the output predictor depends on S). The goal of the algorithm is to find h_S that minimizes the error with respect to the unknown $\mathcal{D}$ and f .

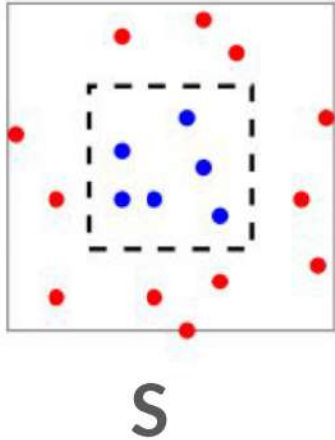
Since the learner does not know what $\mathcal{D}$ and f are, the true error is not directly available to the learner. A useful notion of error that can be calculated by the learner is the *training error* – the error the classifier incurs over the training sample:

$$L_S(h) \stackrel{\text{def}}{=} \frac{|\{i \in [m] : h(x_i) \neq y_i\}|}{m}, \quad (2.2)$$

where $[m] = \{1, \dots, m\}$.

The terms *empirical error* and *empirical risk* are often used interchangeably for this error.

Empirical Risk Minimization: Overfitting



$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases}$$

Empirical-minimum-cost hypothesis

What is the true error of this hypothesis?

Empirical Risk Minimization: Overfitting

$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases}$$

Exercise

-
1. **Overfitting of polynomial matching:** We have shown that the predictor defined in Equation (2.3) leads to overfitting. While this predictor seems to be very unnatural, the goal of this exercise is to show that it can be described as a thresholded polynomial. That is, show that given a training set $S = \{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^m \subseteq (\mathbb{R}^d \times \{0, 1\})^m$, there exists a polynomial p_S such that $h_S(\mathbf{x}) = 1$ if and only if $p_S(\mathbf{x}) \geq 0$, where h_S is as defined in Equation (2.3). It follows that learning the class of all thresholded polynomials using the ERM rule may lead to overfitting.

ERM with Inductive Bias

How to guarantee that ERM does not overfit?

Good performance w.r.to Training → High chance of good performance over the true data distribution

- Apply ERM learning rule over a restricted search space
- Formally, the learner should choose in advance (before seeing the data) a set of predictors.
- This set is called a hypothesis class and is denoted by H .
- By restricting the learner to choosing a predictor from H , we bias it toward a particular set of predictors.

ERM with Inductive Bias

How to guarantee that ERM does not overfit?

Good performance w.r.to Training → High chance of good performance over the true data distribution

- This set is called a hypothesis class and is denoted by H .
- By restricting the learner to choosing a predictor from H , we bias it toward a particular set of predictors.
- This bias is often called “Inductive Bias”
- The choice of such a restriction is determined before the learner sees the training data,
- Thus, it should ideally be based on some prior knowledge about the problem to be learned.

$$\text{ERM}_{\mathcal{H}}(S) \in \underset{h \in \mathcal{H}}{\operatorname{argmin}} L_S(h),$$

ERM with Inductive Bias:

Finite Hypothesis classes

- The simplest type of restriction on a class is imposing an upper bound on its size
- if H is a finite class then ERM_H will not overfit, provided it is based on a sufficiently large training sample
- This size requirement will depend on the size of H
- Limiting the learner to prediction rules within some finite hypothesis class may be considered as a reasonably mild restriction.
- For example, H can be the set of all predictors that can be implemented by a C++ program written in at most 10^9 bits of code.

ERM with Inductive Bias:

Finite Hypothesis classes

- The simplest type of restriction on a class is imposing an upper bound on its size
- if H is a finite class then ERM_H will not overfit, provided it is based on a sufficiently large training sample
- This size requirement will depend on the size of H
- Limiting the learner to prediction rules within some finite hypothesis class may be considered as a reasonably mild restriction.
- For example, H can be the set of all predictors that can be implemented by a C++ program written in at most 10^9 bits of code.

ERM with Inductive Bias:

Finite Hypothesis classes

- Let us now analyze the performance of the ERM_H learning rule assuming that H is a finite class.
- We make the following simplifying assumption
$$h_S \in \underset{h \in \mathcal{H}}{\operatorname{argmin}} L_S(h).$$

DEFINITION 2.1 (The Realizability Assumption) There exists $h^* \in \mathcal{H}$ s.t. $L_{(\mathcal{D}, f)}(h^*) = 0$. Note that this assumption implies that with probability 1 over random samples, S , where the instances of S are sampled according to $\mathcal{D}$ and are labeled by f , we have $L_S(h^*) = 0$.

The realizability assumption implies that for every ERM hypothesis we have that³ $L_S(h_S) = 0$. However, we are interested in the *true* risk of h_S , $L_{(\mathcal{D}, f)}(h_S)$, rather than its empirical risk.

ERM with Inductive Bias:

Finite Hypothesis classes

- Let us now analyze the performance of the ERM_H learning rule assuming that H is a finite class.
- We make the following simplifying assumption
$$h_S \in \underset{h \in \mathcal{H}}{\operatorname{argmin}} L_S(h).$$

DEFINITION 2.1 (The Realizability Assumption) There exists $h^* \in \mathcal{H}$ s.t. $L_{(\mathcal{D}, f)}(h^*) = 0$. Note that this assumption implies that with probability 1 over random samples, S , where the instances of S are sampled according to $\mathcal{D}$ and are labeled by f , we have $L_S(h^*) = 0$.

The realizability assumption implies that for every ERM hypothesis we have that³ $L_S(h_S) = 0$. However, we are interested in the *true* risk of h_S , $L_{(\mathcal{D}, f)}(h_S)$, rather than its empirical risk.

ERM with Inductive Bias:

Finite Hypothesis classes

- We also need assumption on the relationship between D and S $h_S \in \operatorname{argmin}_{h \in \mathcal{H}} L_S(h)$.
- **The i.i.d. assumption:** The examples in the training set are independently and identically distributed (i.i.d.) according to the distribution $\mathcal{D}$. That is, every x_i in S is freshly sampled according to $\mathcal{D}$ and then labeled according to the labeling function, f . We denote this assumption by $S \sim \mathcal{D}^m$ where m is the size of S , and $\mathcal{D}^m$ denotes the probability over m -tuples induced by applying $\mathcal{D}$ to pick each element of the tuple independently of the other members of the tuple.

Intuitively, the training set S is a window through which the learner gets partial information about the distribution $\mathcal{D}$ over the world and the labeling function, f . The larger the sample gets, the more likely it is to reflect more accurately the distribution and labeling used to generate it.

ERM with Inductive Bias:

Finite Hypothesis classes

$$h_S \in \underset{h \in \mathcal{H}}{\operatorname{argmin}} L_S(h).$$

- We also need to address the **probability** to sample a training set for which $L_{(D,f)}(h_S)$ is not too large.
- We denote the probability of getting a nonrepresentative sample by δ , and call $(1 - \delta)$ the confidence parameter of our prediction.
- We introduce another parameter for the quality of prediction, the **accuracy** parameter
- We interpret the event $L_{(D,f)}(h_S) > \epsilon$ as a failure of the learner, while if $L_{(D,f)}(h_S) \leq \epsilon$ we view the output of the algorithm as an approximately correct predictor.

ERM with Inductive Bias:

Finite Hypothesis classes

$$h_S \in \operatorname{argmin}_{h \in \mathcal{H}} L_S(h).$$

- We would like to upper bound

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D}, f)}(h_S) > \epsilon\}).$$

- “The probability of drawing a sequence of **m** training points such that the model trained on those points has a **true generalization error** greater than **ϵ**.”

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\})$.

Let $\mathcal{H}_B$ be the set of “bad” hypotheses, that is,

$$\mathcal{H}_B = \{h \in \mathcal{H} : L_{(\mathcal{D},f)}(h) > \epsilon\}.$$

In addition, let

$$M = \{S|_x : \exists h \in \mathcal{H}_B, L_S(h) = 0\}$$

Here M is the set of misleading samples. Namely, for every $S|_x \in M$, there is a “bad” hypothesis, $h \in \mathcal{H}_B$, that looks like a “good” hypothesis on $S|_x$.

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\})$.
- i.e., we would like to bound the probability of the event $L_{(\mathcal{D},f)}(h_S) > \epsilon$.
- But, since the realizability assumption implies that $L_S(h_S) = 0$, it follows that the event $L_{(\mathcal{D},f)}(h_S) > \epsilon$ can only happen if for some $h \in H_B$ we have $L_S(h) = 0$.
- In other words, this event will only happen if our sample is in the set of misleading samples, M .
- Thus we have shown $\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\} \subseteq M$.

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\})$.

- We have formally shown

$$\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\} \subseteq M .$$

Note that we can rewrite M as

$$M = \bigcup_{h \in \mathcal{H}_B} \{S|_x : L_S(h) = 0\}. \quad (2.5)$$

Hence,

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \mathcal{D}^m(M) = \mathcal{D}^m(\bigcup_{h \in \mathcal{H}_B} \{S|_x : L_S(h) = 0\}). \quad (2.6)$$

Next, we upper bound the right-hand side of the preceding equation using the *union bound* – a basic property of probabilities.

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\})$.

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \mathcal{D}^m(M) = \mathcal{D}^m(\cup_{h \in \mathcal{H}_B} \{S|_x : L_S(h) = 0\}). \quad (2.6)$$

Next, we upper bound the right-hand side of the preceding equation using the *union bound* – a basic property of probabilities.

LEMMA 2.2 (Union Bound) *For any two sets A, B and a distribution $\mathcal{D}$ we have*

$$\mathcal{D}(A \cup B) \leq \mathcal{D}(A) + \mathcal{D}(B).$$

Applying the union bound to the right-hand side of Equation (2.6) yields

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \sum_{h \in \mathcal{H}_B} \mathcal{D}^m(\{S|_x : L_S(h) = 0\}). \quad (2.7)$$

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \sum_{h \in \mathcal{H}_B} \mathcal{D}^m(\{S|_x : L_S(h) = 0\}).$ (2.7)
- The event $L_S(h) = 0$ is equivalent to the event $\forall i, h(x_i) = f(x_i)$
- Since the examples in the training set are sampled i.i.d. We get that

$$\begin{aligned}\mathcal{D}^m(\{S|_x : L_S(h) = 0\}) &= \mathcal{D}^m(\{S|_x : \forall i, h(x_i) = f(x_i)\}) \\ &= \prod_{i=1}^m \mathcal{D}(\{x_i : h(x_i) = f(x_i)\}).\end{aligned}\quad (2.8)$$

For each individual sampling of an element of the training set we have

$$\mathcal{D}(\{x_i : h(x_i) = y_i\}) = 1 - L_{(\mathcal{D},f)}(h) \leq 1 - \epsilon,$$

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \sum_{h \in \mathcal{H}_B} \mathcal{D}^m(\{S|_x : L_S(h) = 0\}).$ (2.7)

$$\begin{aligned}\mathcal{D}^m(\{S|_x : L_S(h) = 0\}) &= \mathcal{D}^m(\{S|_x : \forall i, h(x_i) = f(x_i)\}) \\ &= \prod_{i=1}^m \mathcal{D}(\{x_i : h(x_i) = f(x_i)\}).\end{aligned}\quad (2.8)$$

For each individual sampling of an element of the training set we have

$$\mathcal{D}(\{x_i : h(x_i) = y_i\}) = 1 - L_{(\mathcal{D},f)}(h) \leq 1 - \epsilon,$$

where the last inequality follows from the fact that $h \in \mathcal{H}_B$. Combining the previous equation with Equation (2.8) and using the inequality $1 - \epsilon \leq e^{-\epsilon}$ we obtain that for every $h \in \mathcal{H}_B$,

$$\mathcal{D}^m(\{S|_x : L_S(h) = 0\}) \leq (1 - \epsilon)^m \leq e^{-\epsilon m}. \quad (2.9)$$

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \sum_{h \in \mathcal{H}_B} \mathcal{D}^m(\{S|_x : L_S(h) = 0\}).$ (2.7)

$$\mathcal{D}^m(\{S|_x : L_S(h) = 0\}) \leq (1 - \epsilon)^m \leq e^{-\epsilon m}. \quad (2.9)$$

Combining this equation with Equation (2.7) we conclude that

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq |\mathcal{H}_B| e^{-\epsilon m} \leq |\mathcal{H}| e^{-\epsilon m}.$$

ERM with Inductive Bias:

Finite Hypothesis classes

- We would like to upper bound $\mathcal{D}^m(\{S|x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \sum_{h \in \mathcal{H}_B} \mathcal{D}^m(\{S|x : L_S(h) = 0\}). \quad (2.7)$

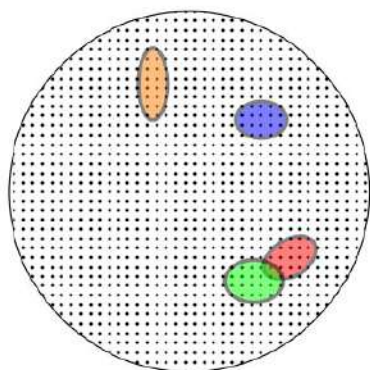


Figure Each point in the large circle represents a possible m -tuple of instances. Each colored oval represents the set of “misleading” m -tuple of instances for some “bad” predictor $h \in \mathcal{H}_B$. The ERM can potentially overfit whenever it gets a misleading training set S . That is, for some $h \in \mathcal{H}_B$ we have $L_S(h) = 0$. Equation (2.9) guarantees that for each individual bad hypothesis, $h \in \mathcal{H}_B$, at most $(1 - \epsilon)^m$ -fraction of the training sets would be misleading. In particular, the larger m is, the smaller each of these colored ovals becomes. The union bound formalizes the fact that the area representing the training sets that are misleading with respect to some $h \in \mathcal{H}_B$ (that is, the training sets in M) is at most the sum of the areas of the colored ovals. Therefore, it is bounded by $|\mathcal{H}_B|$ times the maximum size of a colored oval. Any sample S outside the colored ovals cannot cause the ERM rule to overfit.

ERM with Inductive Bias:

Finite Hypothesis classes

- Can you prove the following?

Let $\mathcal{H}$ be a finite hypothesis class. Let $\delta \in (0, 1)$ and $\epsilon > 0$ and let m be an integer that satisfies

$$m \geq \frac{\log(|\mathcal{H}|/\delta)}{\epsilon}.$$

Then, for any labeling function, f , and for any distribution, $\mathcal{D}$, for which the realizability assumption holds (that is, for some $h \in \mathcal{H}$, $L_{(\mathcal{D},f)}(h) = 0$), with probability of at least $1 - \delta$ over the choice of an i.i.d. sample S of size m , we have that for every ERM hypothesis, h_S , it holds that

$$L_{(\mathcal{D},f)}(h_S) \leq \epsilon.$$

Exercise

2. Let $\mathcal{H}$ be a class of binary classifiers over a domain $\mathcal{X}$. Let $\mathcal{D}$ be an unknown distribution over $\mathcal{X}$, and let f be the target hypothesis in $\mathcal{H}$. Fix some $h \in \mathcal{H}$. Show that the expected value of $L_S(h)$ over the choice of $S|_x$ equals $L_{(\mathcal{D},f)}(h)$, namely,

$$\mathbb{E}_{S|_x \sim \mathcal{D}^m} [L_S(h)] = L_{(\mathcal{D},f)}(h).$$

Exercise

3. **Axis aligned rectangles:** An axis aligned rectangle classifier in the plane is a classifier that assigns the value 1 to a point if and only if it is inside a certain rectangle. Formally, given real numbers $a_1 \leq b_1, a_2 \leq b_2$, define the classifier $h_{(a_1, b_1, a_2, b_2)}$ by

$$h_{(a_1, b_1, a_2, b_2)}(x_1, x_2) = \begin{cases} 1 & \text{if } a_1 \leq x_1 \leq b_1 \text{ and } a_2 \leq x_2 \leq b_2 \\ 0 & \text{otherwise} \end{cases}. \quad (2.10)$$

The class of all axis aligned rectangles in the plane is defined as

$$\mathcal{H}_{\text{rec}}^2 = \{h_{(a_1, b_1, a_2, b_2)} : a_1 \leq b_1, \text{ and } a_2 \leq b_2\}.$$

Note that this is an infinite size hypothesis class. Throughout this exercise we rely on the realizability assumption.

1. Let A be the algorithm that returns the smallest rectangle enclosing all positive examples in the training set. Show that A is an ERM.

Exercise

2. Show that if A receives a training set of size $\geq \frac{4 \log(4/\delta)}{\epsilon}$ then, with probability of at least $1 - \delta$ it returns a hypothesis with error of at most ϵ .

Hint: Fix some distribution $\mathcal{D}$ over $\mathcal{X}$, let $R^* = R(a_1^*, b_1^*, a_2^*, b_2^*)$ be the rectangle that generates the labels, and let f be the corresponding hypothesis. Let $a_1 \geq a_1^*$ be a number such that the probability mass (with respect to $\mathcal{D}$) of the rectangle $R_1 = R(a_1^*, a_1, a_2^*, b_2^*)$ is exactly $\epsilon/4$. Similarly, let b_1, a_2, b_2 be numbers such that the probability masses of the rectangles $R_2 = R(b_1, b_1^*, a_2^*, b_2^*)$, $R_3 = R(a_1^*, b_1^*, a_2^*, a_2)$, $R_4 = R(a_1^*, b_1^*, b_2, b_2^*)$ are all exactly $\epsilon/4$. Let $R(S)$ be the rectangle returned by A . See illustration in Figure 2.2.

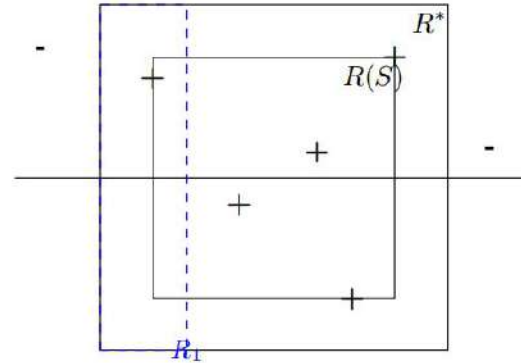


Figure 2.2 Axis aligned rectangles.

Exercise

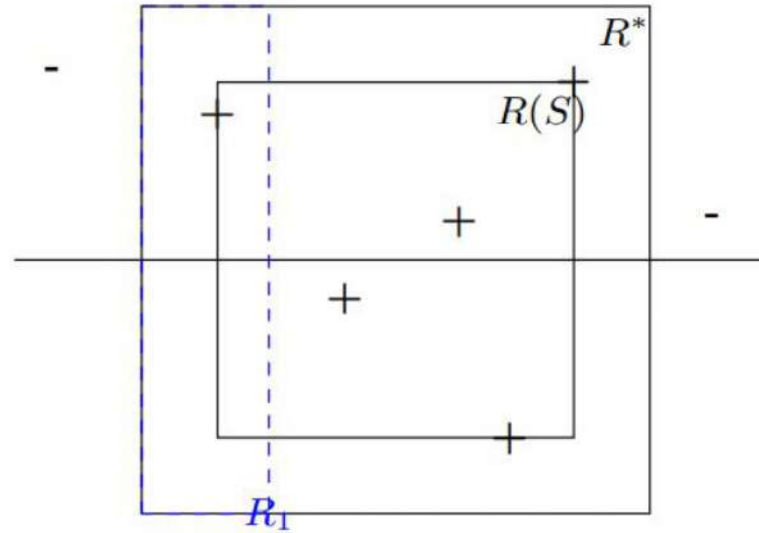


Figure 2.2 Axis aligned rectangles.

- Show that $R(S) \subseteq R^*$.
- Show that if S contains (positive) examples in all of the rectangles R_1, R_2, R_3, R_4 , then the hypothesis returned by A has error of at most ϵ .
- For each $i \in \{1, \dots, 4\}$, upper bound the probability that S does not contain an example from R_i .
- Use the union bound to conclude the argument.

Exercise

- Show that $R(S) \subseteq R^*$.
 - Show that if S contains (positive) examples in all of the rectangles R_1, R_2, R_3, R_4 , then the hypothesis returned by A has error of at most ϵ .
 - For each $i \in \{1, \dots, 4\}$, upper bound the probability that S does not contain an example from R_i .
 - Use the union bound to conclude the argument.
3. Repeat the previous question for the class of axis aligned rectangles in $\mathbb{R}^d$.
 4. Show that the runtime of applying the algorithm A mentioned earlier is polynomial in $d, 1/\epsilon$, and in $\log(1/\delta)$.

Feedback

Please share your valuable feedback .
Form link: bit.ly/feedback_mnm

Acknowledgement

- Chapter 2: Shalev-Shwartz, S., & Ben-David, S. (2014). Understanding machine learning: From theory to algorithms. Cambridge University Press
- Chapter 18 (3rd ed.)/19(4th ed.) (AIAMA) (Textbook)
- Lecture Slides of Sukarna Barua sir, Associate Professor, CSE, BUET.
- Lecture Slides of Dr. Muhammad Ali Nayeem sir, Assistant Professor, CSE, BUET.
- And many other online resources including LLMs.